

AI Streaming : From LLM Tokens to a Live Chat Experience

AI Streaming — From LLM Tokens to a Live Chat Experience

When we use ChatGPT or another modern AI application, the answer usually appears gradually instead of suddenly appearing after several seconds.

It feels as if the AI is **typing in real time**.

That experience is created using **streaming**.

But to understand streaming properly, we first need to understand how an LLM generates text.

1. How an LLM Generates Text

Suppose we give an LLM:

```
The capital of India is
```

The model does not generate the complete answer in one operation:

```
The capital of India is New Delhi.
```

Instead, generation happens incrementally.

Conceptually:

```
The capital of India is
↓
Predict next token
↓
New
```

Now the sequence becomes:

```
The capital of India is New
```

The model runs next-token prediction again:

```
The capital of India is New
↓
Predict next token
↓
Delhi
```

Then again:

```
The capital of India is New Delhi
↓
Predict next token
↓
.
```

So text generation is naturally sequential:

```
Token 1
↓
Token 2
↓
Token 3
↓
Token 4
↓
...
```

A more complete mental model is:

```
Prompt
↓
Calculate next-token probabilities
↓
Choose next token
↓
```

Add token to sequence
↓
Run prediction again
↓
Choose next token
↓
...

This incremental nature of LLM generation is the foundation of AI streaming.

2. Generation and Delivery Are Different Things

Two independent processes are happening in an AI application.

Generation

This is what the model is doing.

Spring
↓
Boot
↓
is
↓
a
↓
framework
↓
...

The model generates output progressively.

Delivery

This is how that generated output travels through our application.

AI Provider
↓
Backend

↓
Browser

Streaming mainly changes **how generated output is delivered to the user**.

It does not fundamentally change the autoregressive way in which the model generates text.

3. Non-Streaming Response

Consider a normal AI request:

```
USER
|
| "Explain Spring Boot"
▼
BACKEND
|
▼
LLM
|
| generates complete answer
▼
BACKEND
|
| complete response
▼
USER
```

The model may generate:

```
Spring
Boot
is
a
framework
...
```

But the backend waits until the complete response is available.

Only then does it return:

"Spring Boot is a framework..."

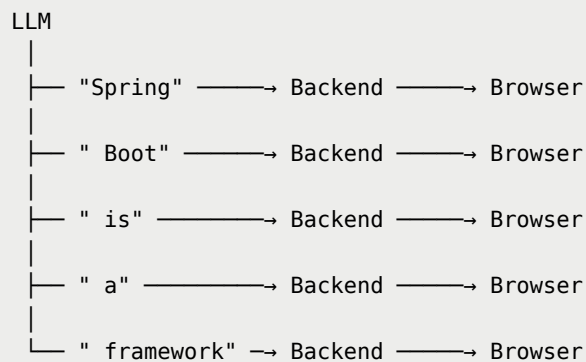
From the user's perspective:

```
Send
↓
.....
.....
.....
↓
Entire answer appears
```

If generation takes eight seconds, the user may stare at an empty interface for most of those eight seconds.

4. Streaming Response

With streaming, generated output is forwarded as it becomes available.



The user might first see:

Spring

Then:

Spring Boot

Then:

Spring Boot is

Eventually:

Spring Boot is a framework...

The complete answer might still require eight seconds.

The important difference is that useful output may begin appearing after only one second.

Streaming does not necessarily make the model finish faster. It makes the application feel faster because the user receives output before generation is complete.

5. Time to First Token vs Total Response Time

Suppose a response behaves like this:

0 sec	User clicks Send
1 sec	First output arrives
2 sec	More output
3 sec	More output
...	
8 sec	Complete answer

There are two different latency measurements here.

Time to First Token / First Output

Approximately:

1 second

This measures how quickly the user begins seeing generated output.

Total Generation Time

Approximately:

8 seconds

This measures how long the complete response takes.

Without streaming:

|----- 8 sec -----|
ANSWER

With streaming:

--1 sec--| Spring
 Boot
 makes
 building
 ...
 COMPLETE at 8 sec

The total generation time may remain similar, but the **perceived latency** is dramatically lower.

That is one of the major reasons streaming is so useful for conversational interfaces.

6. Input Processing Happens Before Output Generation

The model usually cannot generate the first output token immediately.

Before generating output, it first needs to process the input context.

The context may contain:

```
System Prompt
+
Previous Conversation
+
Current User Message
```

Conceptually:

```
System + History + User Message
      ↓
    Process the context
      ↓
-----
      ↓
    Generate output 1
      ↓
    Generate output 2
      ↓
    Generate output 3
      ↓
      ...
```

So there are broadly two phases:

```
Input processing
      ↓
Output generation
```

Streaming becomes useful once output generation begins.

7. Tokens Are Not the Same as Streamed Chunks

An important distinction:

One network chunk does not necessarily equal one LLM token.

The model may internally generate:

```
T1 → T2 → T3 → T4 → T5 → T6
```

But the provider might expose them as:

Chunk 1	Chunk 2	Chunk 3
T1 T2	T3	T4 T5 T6

A streamed piece might contain:

```
"Spring"
```

or:

```
"Spring Boot"
```

or:

```
"Spring Boot makes"
```

The exact grouping depends on the provider, framework, networking layer, buffering and protocol.

For application development, we normally don't need to care about the exact token boundaries.

The useful mental model is simply:

```
small piece  
small piece  
small piece  
small piece  
...
```

The model generates tokens incrementally, while APIs expose generated output as chunks or deltas.

8. What Is API Streaming?

A conventional HTTP API often behaves like this:

```
REQUEST  
↓  
SERVER WORKS  
↓  
SERVER FINISHES  
↓  
RESPONSE
```

For example:

```
GET /users/10
```

The server returns one completed response:

```
{  
  "id": 10,  
  "name": "Rohit"  
}
```

AI responses are different because the result naturally develops over time.

Instead, we want:

REQUEST
↓
Connection remains open
↓
data
↓
data
↓
data
↓
data
↓
DONE

That is streaming.

9. Streaming Is Still an HTTP Response

We often imagine HTTP as:

Request → Response → Connection finished

Streaming does not mean abandoning HTTP.

The server can begin an HTTP response without waiting to construct the entire response body.

Instead of:

HTTP Response

[COMPLETE BODY]

we can have:

HTTP Response begins

chunk
chunk
chunk

```
chunk
chunk

HTTP Response ends
```

The response body arrives progressively.

10. SSE and WebSockets

There are different ways to build real-time communication.

Two common concepts are:

```
SSE – Server-Sent Events

WebSockets
```

Server-Sent Events

SSE stands for **Server-Sent Events**.

Its primary direction is:

```
SERVER
|
| event
▼
CLIENT

SERVER
|
| event
▼
CLIENT
```

A single HTTP connection can deliver multiple events:

```
data: Spring

data: Boot
```

```
data: is  
data: awesome
```

The client can progressively build:

```
Spring Boot is awesome
```

The important characteristic is:

```
SERVER → CLIENT
```

WebSockets

WebSockets provide bidirectional communication:

```
CLIENT ↔ SERVER
```

Both sides can continuously send messages over the same connection.

For simple AI answer streaming, WebSockets are usually unnecessary.

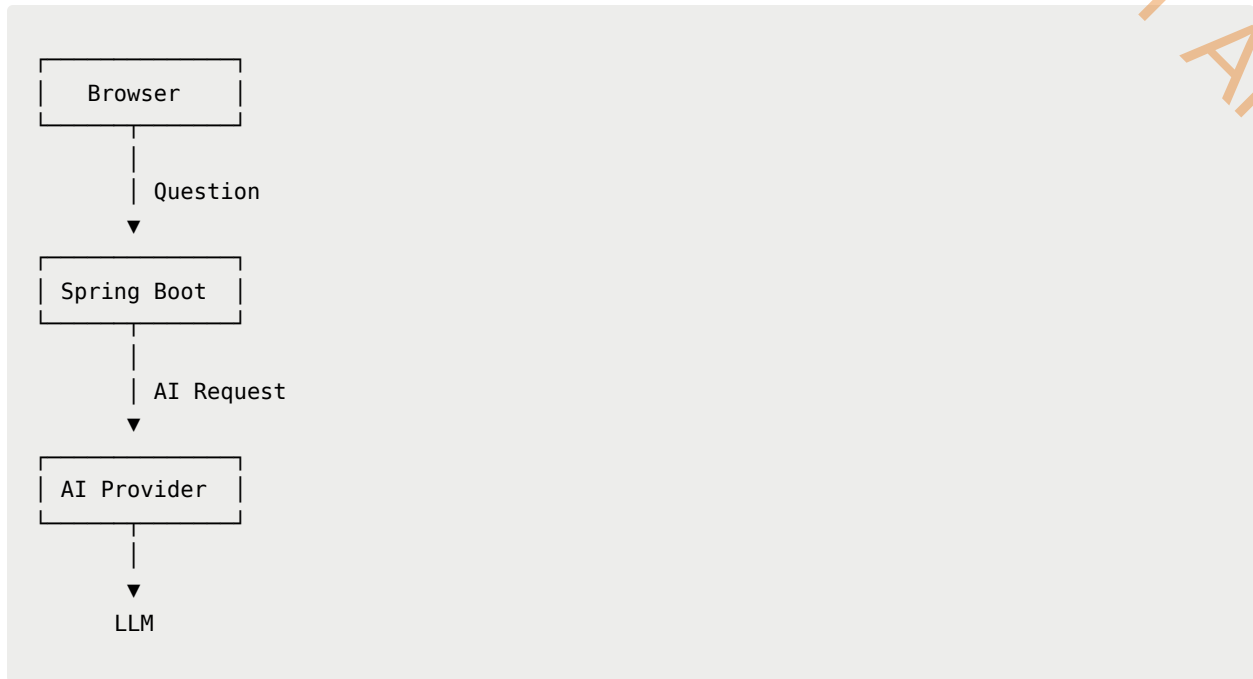
Once the user sends a prompt, most continuous data flows in one direction:

```
Client → Question  
  
Server → answer chunk  
Server → answer chunk  
Server → answer chunk  
Server → answer chunk
```

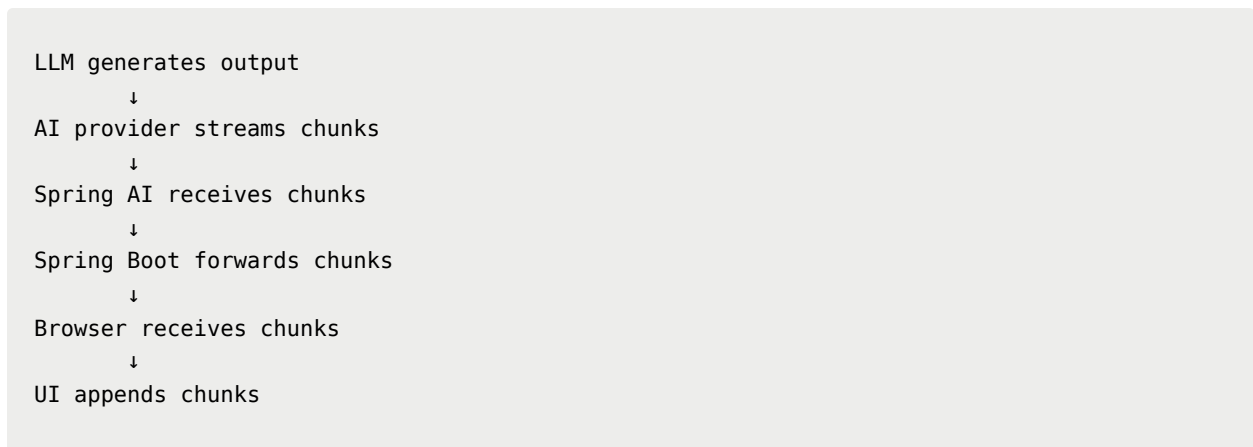
WebSockets can still be useful for more interactive real-time applications, but they are not required just to stream an AI response.

11. The Complete Streaming Architecture

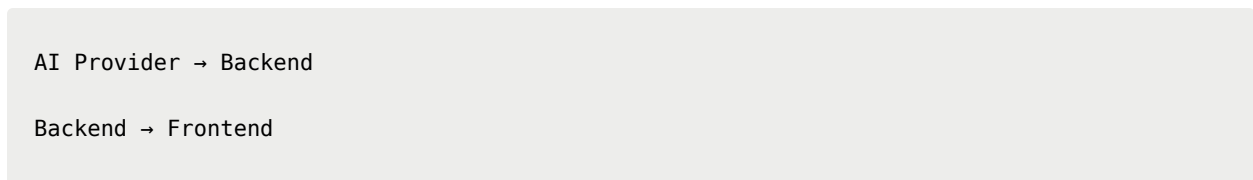
A typical application looks like this:



While generating the response:



From our application's perspective, there are really two important streams:



Both must remain streamed.

12. Streaming Must Survive the Entire Pipeline

Suppose the AI provider sends chunks to our backend:

```
AI Provider
  ↓
chunk
chunk
chunk
chunk
  ↓
Backend
```

If the backend collects everything:

```
chunk
chunk
chunk
chunk

↓ collect

Complete String
```

and only then sends it to the browser, the user does not experience streaming.

Similarly, the backend could stream perfectly, but if JavaScript waits for the complete body before displaying anything, streaming disappears again.

For visible streaming, the whole pipeline must preserve it:

```
LLM
  ↓
AI Provider
  ↓
Spring AI
  ↓
Controller
  ↓
HTTP Response
  ↓
JavaScript
```

↓
DOM

Streaming is an end-to-end property of the application, not just an LLM feature.

13. Spring AI: `.call()` vs `.stream()`

A normal Spring AI call might look like:

```
public String chat(String message) {  
  
    return chatClient.prompt()  
        .user(message)  
        .call()  
        .content();  
}
```

Let's understand the important pieces.

`chatClient.prompt()`

Starts constructing the prompt.

`.user(message)`

Adds the user's message.

`.call()`

Requests the normal non-streaming interaction.

Finally:

`.content()`

returns the generated content.

The return type is:

```
String
```

Conceptually:

```
String answer;
```

One completed value.

14. Switching to `.stream()`

For streaming, instead of:

```
.call()
```

we use:

```
.stream()
```

For example:

```
public Flux<String> chat(String message) {  
  
    return chatClient.prompt()  
        .user(message)  
        .stream()  
        .content();  
}
```

Now the interesting change is not just:

```
.call()  
↓
```

```
.stream()
```

The important change is the return type:

```
String
```

becomes:

```
Flux<String>
```

15. Understanding `Flux<String>`

A normal:

```
String
```

means:

| Eventually give me one String.

For example:

```
"Hello Rohit, how can I help?"
```

A:

```
Flux<String>
```

conceptually means:

| Give me zero, one or many Strings over time.

For example:

```
time →
"Hello"
  " Rohit"
    ", how"
      " can"
        " I help?"
```

Think of a `String` like a box:

```

| Hello Rohit, how can I help? |

```

Think of a `Flux<String>` more like a conveyor belt:

```

→ ["Hello"]
  → [" Rohit"]
    → [", how"]
      → [" can"]
        → [" I help?"]

```

For understanding this AI streaming example, that mental model is enough.

Different languages use different abstractions for the same general idea.

For example:

```

Java / Reactor → Flux
JavaScript     → ReadableStream
Python        → AsyncIterator

```

The syntax changes, but the underlying concept remains:

Values become available progressively over time.

16. Spring AI Streaming Service

Our service can remain very small:

```
@Service
public class ChatService {

    private final ChatClient chatClient;

    public ChatService(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    public Flux<String> chat(String message) {

        return chatClient.prompt()
            .user(message)
            .stream()
            .content();
    }
}
```

Instead of waiting for one complete String, the service exposes pieces of generated text as they arrive.

17. Streaming from Spring Boot to the Browser

There are multiple ways to expose streaming over HTTP.

For the Tomato chatbot, the simplest approach is **raw text streaming**.

```
@PostMapping(
    value = "/api/chat",
```

```

        consumes = MediaType.TEXT_PLAIN_VALUE,
        produces = MediaType.TEXT_PLAIN_VALUE
    )
    public Flux<String> chat(@RequestBody String message) {

        return chatService.chat(message);
    }

```

The important part is that the controller returns:

```
Flux<String>
```

instead of:

```
String
```

Spring can progressively write those emitted pieces into the HTTP response.

18. What About **text/event-stream** ?

If we deliberately use SSE, the controller may produce:

```
MediaType.TEXT_EVENT_STREAM_VALUE
```

which corresponds roughly to:

```
Content-Type: text/event-stream
```

That is useful when we want actual Server-Sent Events.

However, **streaming itself does not require SSE.**

For our simple Tomato chatbot, raw text is enough:

```
produces = MediaType.TEXT_PLAIN_VALUE
```

So it is useful to separate these concepts:

```
Streaming = data arrives progressively
```

```
SSE = one particular protocol/format for delivering events
```

19. Conversation History Creates Another Problem

Suppose our chatbot already maintains conversation history:

```
List<Message> history = new ArrayList<>();
```

A user sends:

```
My name is Rohit
```

We add:

```
history.add(new UserMessage("My name is Rohit"));
```

With non-streaming code, the assistant response eventually becomes:

```
String response = "...";
```

Then we simply store:

```
history.add(new AssistantMessage(response));
```

That works because the assistant response is already one complete String.

Streaming changes this.

20. Don't Store Every Chunk as a Separate Message

With streaming, the response may arrive as:

```
"My"  
" name"  
" is"  
" ChatGPT"
```

We should not create:

```
AssistantMessage("My")  
AssistantMessage(" name")  
AssistantMessage(" is")  
AssistantMessage(" ChatGPT")
```

Those are transport chunks.

Logically, the model generated **one assistant message**:

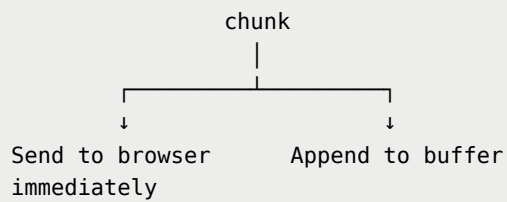
```
"My name is ChatGPT"
```

Our conversation history should therefore contain:

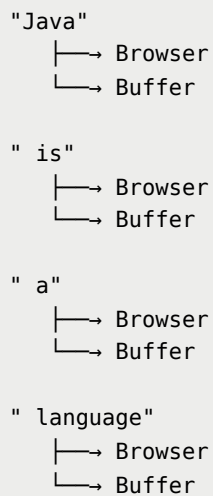
```
UserMessage(...)  
  
AssistantMessage("My name is ChatGPT")
```

21. Stream to the Browser and Buffer at the Same Time

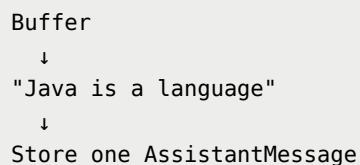
Every chunk should perform two jobs:



For example:



When generation completes:



This gives us both:

- Real-time output for the user
- Correct conversation history for future prompts

22. Streaming with Conversation History

A simple teaching implementation could look like:

```
public Flux<String> chat(String userMessage) {  
  
    history.add(new UserMessage(userMessage));  
  
    StringBuilder fullResponse = new StringBuilder();  
  
    return chatClient.prompt()  
        .messages(history)  
        .stream()  
        .content()  
        .doOnNext(fullResponse::append)  
        .doOnComplete(() -> {  
            history.add(  
                new AssistantMessage(fullResponse.toString()  
            ));  
        });  
}
```

This line:

```
.doOnNext(fullResponse::append)
```

means:

Whenever another chunk arrives, append it to the local buffer.

And:

```
.doOnComplete(...)
```

means:

When the stream finishes successfully, convert the buffered output into one assistant message and save it.

The stream itself continues flowing to the controller and browser.

Production Note

A shared `ArrayList` and mutable conversation history like this are useful for teaching but are not suitable as-is for a real multi-user application.

Production applications need proper conversation isolation and thread-safe state management.

23. Backend Streaming Is Only Half the Work

At this point:

```
AI Provider → Spring Boot
```

is streaming.

But we still need:

```
Spring Boot → Browser
```

to behave the same way.

If the browser waits for the complete HTTP body, the user still sees the entire answer appear at once.

24. The Common Frontend Mistake

Suppose the backend emits:

```
"Spring"  
" Boot"  
" is"
```

```
" a"  
" framework"
```

But the frontend does:

```
const assistantReply = await response.text();
```

Then the browser effectively says:

Read the complete response body, combine everything, convert it to text, and give it to me only after it has finished.

So our architecture becomes:

LLM	✓ streaming
↓	
Spring Boot	✓ streaming
↓	
HTTP	✓ streaming
↓	
Browser	✓ receiving data
↓	
response.text()	✗ waits for complete body
↓	
UI	

One line has removed the visible streaming experience.

25. What Does `await fetch()` Actually Wait For?

Consider:

```
const response = await fetch(API_URL, {  
  method: "POST",  
  headers: {  
    "Content-Type": "text/plain"  
  },  
});
```

```
    body: message  
  });
```

A common misconception is:

`await fetch()` waits until the complete response body has downloaded.

Conceptually, it does not.

The flow is closer to:

```
Browser sends request  
  ↓  
Server receives request  
  ↓  
Server starts HTTP response  
  ↓  
Response headers arrive  
  ↓  
fetch() resolves
```

At this point:

```
response
```

exists.

But the response body may still be arriving.

Think of it as:

```
HTTP Response  
  
Status: 200  
Headers: ...  
  
Body:  
  ↓  
still arriving...
```

The problem comes when we call:

```
await response.text();
```

That asks the browser to consume the body and give us the completed text.

For normal APIs, that is exactly what we want.

For a live-streaming UI, we want access to the body **while it is arriving**.

26. `response.body`

The actual stream is available through:

```
response.body
```

Conceptually:

```
graph TD
    response --> status
    response --> headers
    response --> body
    body --> ReadableStream
```

A `ReadableStream` represents data that can become available progressively over time.

We can obtain a reader:

```
const reader = response.body.getReader();
```

Conceptually:

```
graph TD
    HTTP[HTTP Response Body] --> ReadableStream
    ReadableStream --> getReader[getReader()]
```

Reader

27. Reading the Stream

We can repeatedly ask the reader for more data:

```
const { value, done } = await reader.read();
```

The result conceptually looks like:

```
{  
  value: ...,  
  done: false  
}
```

Later:

```
{  
  value: ...,  
  done: false  
}
```

Eventually:

```
{  
  value: undefined,  
  done: true  
}
```

So the natural structure becomes:

```
while (true) {  
  
  const { value, done } = await reader.read();
```

```

    if (done) {
        break;
    }

    // Process incoming data
}

```

Now the browser is actively consuming the response while it is still arriving.

28. Why Isn't **value** Already a String?

At the network level, HTTP transfers bytes.

For example:

```

"Spring"
  ↓
UTF-8 Encoding
  ↓
83 112 114 105 110 103
  ↓
Network

```

So:

value

is usually byte data represented as something like:

Uint8Array

Conceptually:

[83, 112, 114, 105, 110, 103]

Our UI needs text.

Therefore we need:

```
Bytes
↓
Decode
↓
Text
```

29. TextDecoder

JavaScript provides `TextDecoder` for this job.

Create it once:

```
const decoder = new TextDecoder();
```

Then decode every incoming piece:

```
const chunk = decoder.decode(value, {
  stream: true
});
```

Conceptually:

```
Uint8Array
↓
TextDecoder
↓
"Spring"
```

Our streaming loop becomes:

```
const reader = response.body.getReader();
const decoder = new TextDecoder();

while (true) {
```

Coder Army

```
const { value, done } = await reader.read();

if (done) {
  break;
}

const chunk = decoder.decode(value, {
  stream: true
});

console.log(chunk);
}
```

The console might gradually show:

```
Spring
Boot
makes
building
Java
applications
easier
```

But again:

A browser chunk is not guaranteed to be one token or one word.

It might receive:

```
"Spring Boot"
```

followed by:

```
" makes building"
```

followed by:

```
" Java applications"
```

The UI should not depend on chunk boundaries.

30. Build One Assistant Bubble, Not One Bubble per Chunk

Suppose the response arrives as:

```
"Spring"  
" Boot"  
" is"  
" great"
```

We should not create:

```
[Spring]  
  
[ Boot]  
  
[ is]  
  
[ great]
```

These are not four assistant messages.

They are four pieces of **one assistant message**.

The correct approach is:

```
Create assistant bubble once  
    ↓  
Append chunk  
    ↓  
Append next chunk  
    ↓  
Append next chunk
```

↓
Complete message

The UI gradually changes from:

Spring

to:

Spring Boot

to:

Spring Boot is

to:

Spring Boot is great

31. Reusing the Existing Chat UI

If our existing helper returns the newly created row:

```
return row;
```

we can create one empty assistant message:

```
const assistantRow = addMessage("assistant", "");
```

Then get its message bubble:

```
const assistantBubble =  
  assistantRow.querySelector(".message");
```

Whenever text arrives:

```
assistantBubble.textContent += chunk;
```

The same bubble keeps growing.

32. Complete Streaming `sendMessage()`

```
async function sendMessage() {  
  
  const message = messageInput.value.trim();  
  
  if (!message) {  
    return;  
  }  
  
  addMessage("user", message);  
  
  messageInput.value = "";  
  messageInput.focus();  
  
  sendButton.disabled = true;  
  
  showTyping();  
  
  let assistantRow = null;  
  let assistantBubble = null;  
  let hasStartedStreaming = false;  
  
  try {
```

```

const response = await fetch(API_URL, {
  method: "POST",
  headers: {
    "Content-Type": "text/plain"
  },
  body: message
});

if (!response.ok) {
  throw new Error("Request failed");
}

const reader = response.body.getReader();
const decoder = new TextDecoder();

while (true) {

  const { value, done } = await reader.read();

  if (done) {
    break;
  }

  const chunk = decoder.decode(value, {
    stream: true
  });

  if (!hasStartedStreaming) {

    hideTyping();

    assistantRow = addMessage(
      "assistant",
      ""
    );
  }
}

```

```

        assistantBubble =
            assistantRow.querySelector(".message");

        hasStartedStreaming = true;
    }

    assistantBubble.textContent += chunk;

    scrollToBottom();
}

const remainingText = decoder.decode();

if (remainingText && assistantBubble) {
    assistantBubble.textContent += remainingText;
}

} catch (error) {

    hideTyping();

    if (assistantRow) {
        assistantRow.remove();
    }

    addMessage(
        "assistant",
        "Sorry, I couldn't connect to Tomato Support. Please try again.",
        "error"
    );

} finally {

    hideTyping();
}

```

```
        sendButton.disabled = false;
    }
}
```

33. What Happens in the Frontend?

When the user sends a message:

```
graph TD
    A[User submits message] --> B[Show typing indicator]
    B --> C[Send HTTP request]
    C --> D[Receive response headers]
    D --> E[Get response.body]
    E --> F[Create stream reader]
    F --> G[Wait for first chunk]
    G --> H[Hide typing indicator]
    H --> I[Create ONE assistant bubble]
    I --> J[Decode chunk]
    J --> K[Append text]
    K --> L[Read next chunk]
    L --> M[Append text]
    M --> N[...]
    N --> O[Stream completes]
```

This produces the familiar ChatGPT-style experience where the answer appears progressively.

34. Why Call `scrollTopBottom()` After Every Chunk?

As the response grows:

```
Line 1  
Line 2  
Line 3  
Line 4  
Line 5  
...
```

the assistant bubble becomes taller.

Without updating the scroll position, newly generated content may move outside the visible area.

Calling:

```
scrollTopBottom();
```

after each chunk keeps the latest generated output visible.

For production applications, we may want smarter behavior so that automatic scrolling stops if the user manually scrolls upward.

For a basic chatbot demo, continuous scrolling is sufficient.

35. Token vs Provider Chunk vs Network Chunk

One of the most useful distinctions in the entire topic is:

```
LLM Token  
≠  
API Provider Chunk  
≠  
HTTP / Network Chunk
```

≠
reader.read() Result

Imagine the model generates:

Token 1 = "Spring"
Token 2 = " Boot"
Token 3 = " is"
Token 4 = " great"

The provider might expose:

"Spring"

" Boot"

" is great"

The backend receives those pieces.

But due to networking or buffering, the browser may receive:

"Spring Boot"

followed by:

" is great"

There is no guarantee that the same chunk boundaries remain intact through every layer.

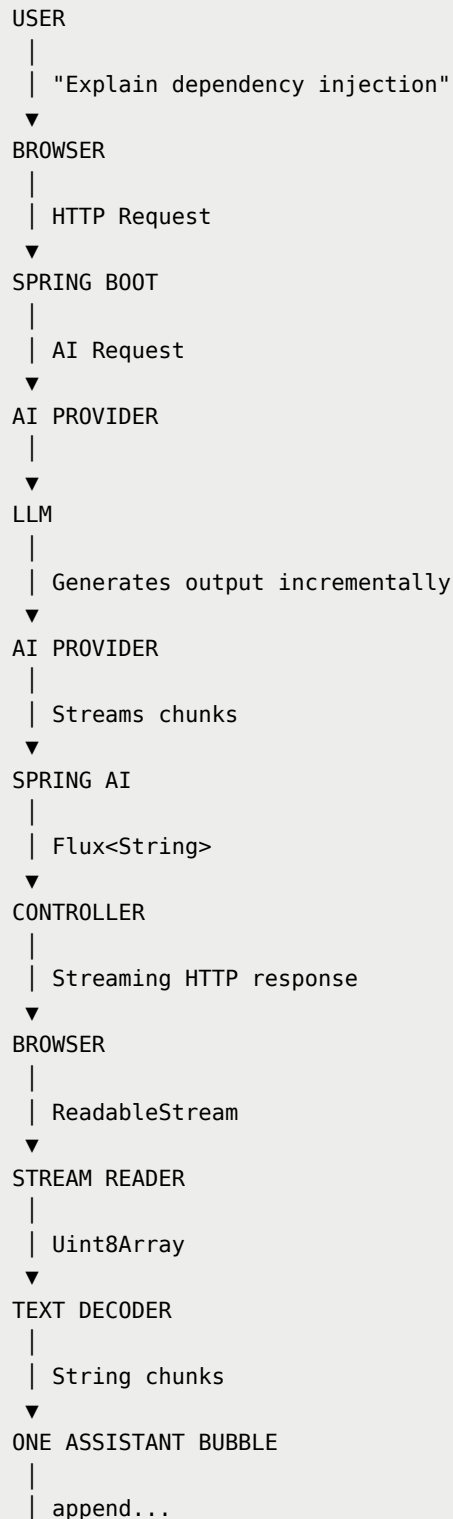
Fortunately, the UI normally does not need to care.

Its responsibility is simply:

Whenever more text becomes available, append it to the current assistant message.

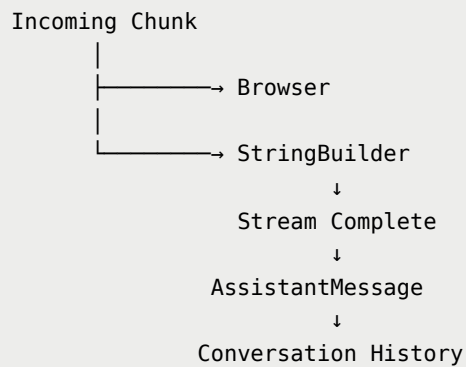
36. Final End-to-End Mental Model

The complete flow is:



```
| append...  
| append...  
▼  
COMPLETE RESPONSE
```

At the same time, the backend may buffer those pieces:



37. The Core Idea

AI streaming becomes much easier to understand once we separate the layers.

The LLM generates tokens incrementally.

The AI provider exposes generated output as chunks or deltas.

Spring AI represents those streamed values using:

```
Flux<String>
```

Spring Boot progressively sends them through the HTTP response.

The browser exposes the incoming response through:

```
response.body
```

which is a:

```
ReadableStream
```

JavaScript reads the incoming bytes, decodes them into text and continuously appends that text to one assistant message.

The entire application therefore behaves like one continuous pipeline:

```
graph TD; A[Generation] --> B[Provider Streaming]; B --> C[Backend Streaming]; C --> D[HTTP Streaming]; D --> E[Browser Streaming]; E --> F[Live UI Rendering];
```

Generation
↓
Provider Streaming
↓
Backend Streaming
↓
HTTP Streaming
↓
Browser Streaming
↓
Live UI Rendering

And that is what creates the familiar experience of an AI assistant appearing to **type its answer in real time**.